
AlphaCast

Sector-Augmented Time-Series Transformers for Financial Forecasting

A Comparative Analysis of Transformer and LSTM Architectures in Sector-Driven Stock Prediction

Viva Cheat Sheet (Interactive)

Paras Beniwal — B.Tech Final Year, Semester-8 Major Project

Tip: this PDF is interactive — click any heading in the Table of Contents to jump to that section. Your PDF reader's sidebar also shows a full outline of every section and every Q&A.

Table of Contents

1. The 30-Second Pitch
2. The Pipeline (Flowchart)
3. What Each Notebook Does
4. Final Numbers (AAPL Results)
5. Likely Viva Questions & Answers
 - 5A. Conceptual Questions (Q1–Q4)
 - 5B. Data & Preprocessing (Q5–Q9)
 - 5C. Model Questions (Q10–Q15)
 - 5D. Evaluation Questions (Q16–Q21)
 - 5E. Trick / Deeper Questions (Q22–Q28)
6. Glossary of Key Terms
7. One-Line Defences for Every Choice

1. The 30-Second Pitch

*"I built **AlphaCast**, a system that forecasts the next 7 days of a stock's closing price. Instead of looking at the stock alone, I also feed the model the stock's most-correlated **sector ETF** as a stable anchor. I then ran a head-to-head comparison between a baseline **LSTM** and a state-of-the-art **Temporal Fusion Transformer**. On AAPL, the Transformer beat the LSTM by **72% on both RMSE and MAPE**, confirming that attention handles non-stationary financial data better than recurrence."*

[↑ back to top](#)

2. The Pipeline (Flowchart)

Read top-to-bottom: raw data flows through 4 notebooks into the live Streamlit dashboard.

```
+-----+ |
data/Stocks/.txt data/ETFs/.txt | | (7195 stocks) (1344 ETFs incl. 9 SPDR sectors) |
+-----+-----+ | | v v
+-----+ | N1 - Data & ETF Matching | | - Load
target stock + 9 sector ETFs | | - Pearson corr on DAILY RETURNS | | - Pick
highest-corr ETF as anchor | | - Save merged CSV |
+-----+ | v
+-----+ | N2 - Feature Engineering | | - MA7,
MA21, RSI, MACD on STOCK + ETF | | - Chronological 80/20 split | | - MinMaxScaler fit
on TRAIN ONLY | | - Sliding windows -> (N, 30, 14) | | targets -> (N, 7) |
+-----+ | v
+-----+ | SAME 3-D WINDOWS | | --- fair fight
--- | +-----+-----+ | | v v +-----+
+-----+ | N3 - LSTM (PyTorch) | | N4 - TFT (darts) | | 2 layers,
hidden=64 | | hidden=32, attn=4 | | MSE + Adam | | MSE + Adam | | Early-stop on val |
| 30 epochs | | RMSE / MAPE in $ | | RMSE / MAPE in $ | | lstm_model.pt | |
transformer_model.pt | +-----+ +-----+ +-----+ | |
+-----+ v +-----+ | app.py
- Streamlit Model Showdown | | - Pick stock from dropdown | | - Live 7-day forecast
from BOTH models | | - Plotly chart: actual + LSTM + TFT | | - Showdown table: RMSE,
MAPE, %improve | +-----+
```

[↑ back to top](#)

3. What Each Notebook Does

#	File	Purpose	Key Output
N1	01_Data_and_ETF_Matching.ipynb	Load stock + 9 sector ETFs, find best ETF by daily-return correlation	_with_.csv + meta.json
N2	02_Feature_Engineering.ipynb	Add MA7, MA21, RSI, MACD on stock & ETF; scale; reshape to 3-D windows	X/y .npy arrays + scalers + config.json
N3	03_Baseline_LSTM.ipynb	2-layer LSTM, train + early-stop, evaluate on test set	lstm_model.pt + lstm_metrics.json
N4	04_Champion_Transformer.ipynb	trains TFT trained on the same windows, sliding-window evaluation	transformer_model.pt + showdown.json
App	app.py	Streamlit dashboard, live 7-day forecast from both models, comparison table	Interactive web app on localhost:8501

[↑ back to top](#)

4. Final Numbers (AAPL Results)

Backtest on the held-out 20% test set (601 sliding 7-day windows):

Model	RMSE (\$)	MAPE (%)	# Parameters
LSTM (Baseline)	40.98	26.95	~54,000
TFT (Champion)	11.43	7.52	~30,000
TFT improvement	+72.10%	+72.10%	—

One-line takeaway: on a stock with a 400× price rise (1984 -> 2017), the Transformer's self-attention adapted to the recent regime, while the LSTM's hidden state stayed dominated by old low-price training data.

[↑ back to top](#)

5. Likely Viva Questions & Answers

Click a question in your reader's sidebar to jump straight to it. All Q&A entries are individually bookmarked.

5A. Conceptual Questions

Q1. What is the core idea of your project?

Predict a stock's 7-day price using both the stock's own history **and** its most-correlated sector ETF as a stabilising anchor, then compare LSTM vs Transformer to see which architecture is better at it.

Q2. Why use a sector ETF instead of just the stock?

A single stock is noisy — earnings, lawsuits, tweets all move it. A sector ETF holds dozens of similar companies, so its price is a much smoother signal of the macro trend. Adding it raises the signal-to-noise ratio.

Q3. Why correlate on daily RETURNS instead of prices?

Two stocks both drifting upward over years look correlated in price even if their day-to-day moves are unrelated — that is spurious correlation. Daily returns (pct_change) strip out long-term drift and measure whether they actually move together day-to-day. That is the right signal for sector membership.

Q4. Why these 9 ETFs only?

They are the SPDR sector ETFs — the canonical 9 buckets the US market is divided into (Tech, Finance, Energy, Health, Consumer-Discretionary, Consumer-Staples, Industrials, Materials, Utilities). Restricting to them keeps the matching interpretable.

[↑ back to top](#)

5B. Data & Preprocessing Questions

Q5. Why is your test set the LAST 20% and not random?

Time-series data has temporal order. If you randomly shuffle, your training set could contain future information about your test set — that is data leakage and inflates accuracy. Chronological split is the only honest evaluation for forecasting.

Q6. Why fit the scaler on TRAIN only?

Same reason — to prevent look-ahead leakage. If the scaler sees test data, the train data gets normalised using statistics that include the future, which gives an unrealistic boost.

Q7. What technical indicators did you use and why?

Four classics: **MA7 / MA21** (short / medium trend, rolling means), **RSI(14)** (overbought/oversold momentum oscillator on a 0–100 scale), and **MACD** (fast EMA minus slow EMA, captures trend reversals). They are standard in finance and the formulas are simple — easy to defend.

Q8. Why a 30-day lookback and 7-day horizon?

30 days is roughly one trading month — enough context to see a trend without being so long the network forgets early information. 7 days is one trading week, the project's stated forecasting target.

Q9. What is the shape of one training sample?

Input $X[i]$ is shape (30, 14) — 30 days x 14 features. Target $y[i]$ is shape (7,) — the next 7 days' Close prices. Stack many of those and you get the 3-D tensor (N, 30, 14) that both models consume.

[↑ back to top](#)

5C. Model Questions

Q10. Explain your LSTM architecture.

2 stacked LSTM layers, hidden size 64, dropout 0.2. We take only the LAST time-step's hidden state (a 64-dim vector summarising the 30-day history) and pass it through a linear layer that outputs 7 numbers — the 7-day forecast.

Q11. Why early stopping?

Saves the model weights at the lowest validation loss seen so far. If validation hasn't improved for 7 epochs, training stops and we restore those best weights. This prevents overfitting to the training set.

Q12. What is a Temporal Fusion Transformer (TFT)?

A 2019 forecasting architecture that combines an LSTM encoder/decoder for local sequence patterns with a multi-head self-attention layer that lets the model dynamically pick which past days matter most for each future prediction. It also natively handles past covariates — perfect for sector augmentation.

Q13. Why is attention better than recurrence here?

An LSTM compresses the entire 30-day history into one fixed-size hidden vector — early days get squashed. Attention computes a weighted sum where the weights are learned per query, so the model can ignore irrelevant days and focus on the most informative ones. For non-stationary data like prices that shift regime, this flexibility is decisive.

Q14. How was your fight FAIR?

Same stock, same sector ETF, same chronological 80/20 split, same scaler, same 30/7 windows, same MSE loss, model capacities in the same ballpark (~54k LSTM params vs ~30k TFT params). Only the architecture differed.

Q15. What is a "past covariate" in darts?

A feature whose value is known up to the present but NOT into the future — like Volume or RSI. The model uses them to inform predictions but doesn't need future values. This is exactly the role of the ETF columns and indicators in our setup.

[↑ back to top](#)

5D. Evaluation Questions

Q16. What are RMSE and MAPE? Why both?

RMSE (Root Mean Squared Error) — average dollar error; punishes big mistakes more (squared).

MAPE (Mean Absolute Percentage Error) — error as a % of true price; comparable across stocks of different price levels. RMSE is absolute, MAPE is relative — together they give a complete picture.

Q17. Why did your LSTM get such a poor MAPE (27%)?

Apple's price went from \$0.42 in 1984 to \$170 in 2017 — a 400x rise. The MinMaxScaler fit on training data caps test prices outside the training range. The LSTM's hidden state is dominated by 1984-era low-price patterns and can't generalise to the high-price test regime. This is a textbook NON-STATIONARITY / regime-shift problem in finance.

Q18. So why did the TFT do so much better on the same data?

Self-attention can dynamically focus on the most recent (and price-relevant) lookback days, ignoring the bulk of out-of-distribution training history. The LSTM has no such flexibility — it propagates everything through its single hidden state.

Q19. How did you ensure both models were evaluated on the SAME windows?

The LSTM produced predictions on 601 sliding test windows. For the TFT I used darts' `historical_forecasts(stride=1, last_points_only=False, retrain=False)` which generates one 7-day forecast per stride-1 step across the test set, then trimmed to the same count. Identical input windows, identical evaluation protocol.

Q20. Could the TFT just be winning because it has fewer parameters and so overfits less?

That is a fair question — and the honest answer reinforces our claim. Fewer parameters usually means LESS capacity, not more. The TFT (~30k params) beats the LSTM (~54k params) despite being smaller, which means the win is architectural, not capacity-driven.

Q21. Why MSE as the loss function?

Standard regression loss for continuous outputs. Minimising MSE is equivalent to maximising likelihood under a Gaussian noise assumption, which is a reasonable first approximation for log-price residuals. Using the same loss on both models keeps the comparison fair.

[↑ back to top](#)

5E. Trick / Deeper Questions

Q22. Can your model predict future prices accurately enough to make money?

Honest answer: no, and that is not the goal. The project is a COMPARATIVE ARCHITECTURE STUDY, not a trading system. Real trading would need transaction costs, slippage, risk management, and probably classification of direction rather than regression on price.

Q23. Why didn't you use yfinance / live data?

For a reproducible viva project I wanted a fixed snapshot — the Kaggle "Huge Stock Market Dataset" gives the same results every time the code is run. Live data would change my numbers between viva rehearsals.

Q24. What is the limitation of MinMaxScaler on long-history data?

It is bounded by the train min/max. If test prices exceed the train range (as with AAPL), scaled values fall outside $[0, 1]$ and the model extrapolates. A common workaround is to predict RETURNS (stationary) instead of absolute prices, or to use a rolling window of recent data only.

Q25. Why didn't you use the Transformer paper's vanilla architecture?

Vanilla Transformers (Vaswani et al., 2017) were designed for NLP, not numeric forecasting. The TFT is purpose-built for time series: it adds gating, variable selection networks, and explicit handling of past / future / static covariates. It is the industry-standard transformer for forecasting.

Q26. How would you improve the project further?

(1) Predict returns instead of absolute prices to handle non-stationarity. (2) Add walk-forward validation across multiple time periods. (3) Train on multiple stocks jointly. (4) Try DLinear / NLinear baselines, which are surprisingly competitive. (5) Add a directional accuracy metric on top of RMSE.

Q27. What is the role of dropout?

Randomly zeroes 20% of activations during training, forcing the network to not rely on any single neuron. Acts as a regulariser and reduces overfitting on small datasets like ours.

Q28. Walk me through what happens when I click "Forecast" in your dashboard.

app.py reads the last 30 days of engineered features for the chosen stock, runs them through the loaded LSTM and TFT, inverse-transforms the scaled predictions back to dollars, builds a Plotly chart with the historical price plus both 7-day forecast lines, and renders the showdown table from the saved metrics JSON.

[↑ back to top](#)

6. Glossary of Key Terms

ETF	Exchange-Traded Fund — a basket of stocks that trades like one stock.
SPDR sector ETFs	The 9 ETFs from State Street that split the US market into sectors (XLK Tech, XLF Finance, XLE Energy, XLV Health, XLY Consumer Disc., XLP Consumer Staples, XLI Industrials, XLB Materials, XLU Utilities).
Pearson correlation	A number between -1 and +1 measuring linear co-movement of two series.
Stationarity	A series whose statistical properties (mean, variance) don't change over time. Stock PRICES are non-stationary; stock RETURNS are roughly stationary.
Lookback / Horizon	Lookback = how many past days the model sees as input. Horizon = how many future days it must predict.
Sliding window	Sampling technique where one window slides one step at a time across the timeline, producing many overlapping training samples.
LSTM	Long Short-Term Memory — a recurrent neural network with gates that decide what to remember/forget.
Self-attention	Mechanism where every position in a sequence computes weighted similarity scores against every other position to decide what to focus on.
TFT	Temporal Fusion Transformer — Google Cloud, 2019. Combines an LSTM with multi-head attention specifically for time-series forecasting.
RMSE	$\sqrt{\text{mean}((\text{pred} - \text{actual})^2)}$. Same units as the target (\$). Bigger errors are weighted more.
MAPE	$\text{mean}(\text{pred} - \text{actual} / \text{actual}) * 100$. A relative error in %.
MinMax scaling	Rescales every feature linearly to [0, 1] using the column min and max. Sensitive to outliers but simple.
Past covariate (darts)	Auxiliary feature known only up to the present. Used as model input but not predicted.
Regime shift	A change in the underlying statistical behaviour of a market — e.g., low-volatility 2017 versus high-volatility 2008. Hard for models that assume stationarity.

[↑ back to top](#)

7. One-Line Defences for Every Choice

If asked "Why this and not something else?", here are crisp answers:

Pandas / NumPy	Industry-standard data tooling, easy to defend.
PyTorch for LSTM	Most flexible deep-learning library; standard for any custom NN.
darts for TFT	Single line to instantiate a production-grade TFT; cuts complexity.
Streamlit + Plotly	Fastest way to build an interactive demo; no JS/HTML needed.
Pearson on returns	Strips spurious trend correlation, isolates true co-movement.
80/20 chronological	Honest time-series evaluation, no leakage.
30-day lookback	One trading month — enough context, not too long.
7-day horizon	One trading week — the project requirement.
MinMaxScaler [0,1]	Plays nicely with sigmoid/tanh activations inside LSTMs.
MSE loss	Standard regression loss; same on both models for fairness.
Adam optimiser	Adaptive learning rate, default choice for time series.
Early stopping	Saves best validation weights, prevents overfitting.
Dropout 0.2	Light regularisation suitable for ~50k-param models.
Same windows for both	The single most important fairness control.
CPU for TFT training	darts uses float64; Apple MPS does not support it.

Good luck with your viva! — AlphaCast project, Sem-8

[↑ back to top](#)